

More design problems

(with a bit more fork-join)

5/21/25

Administrivia

- HW 9 (sorting, heaps, fork-join) due tomorrow night
- Last HW: Create your own design problem
 - We'll go over them on the last day of lecture (week from today)
- No lab on last day of classes
- Final on Monday (6/2) afternoon (1:30-4:30pm)
 - Open notes, closed friends and internet
 - Comprehensive, but weighted toward material since the midterm

Recall: Fork-join in Java

```
class Sum extends RecursiveTask<Integer> {  
    //attributes and constructor  
  
    protected Integer compute(){  
        if(hi - low < THRESHOLD) return simpleSum(array, low, hi);  
        Sum left = new Sum(array, low, (hi+low)/2);  
        Sum right= new Sum(array, (hi+low)/2, hi);  
        left.fork();  
        return right.compute() + left.join();  
    }  
}  
  
sum = ForkJoinPool.commonPool().invoke(new Sum(array, 0, array.length));
```

What if I wanted the minimum instead of the sum?

```
class Sum extends RecursiveTask<Integer> {  
    //attributes and constructor  
  
    protected Integer compute(){  
        if(hi - low < THRESHOLD) return simpleSum(array, low, hi);  
        Sum left = new Sum(array, low, (hi+low)/2);  
        Sum right= new Sum(array, (hi+low)/2, hi);  
        left.fork();  
        return right.compute() + left.join();  
    }  
}  
  
sum = ForkJoinPool.commonPool().invoke(new Sum(array, 0, array.length));
```

What if I wanted the minimum and its index?

```
class Sum extends RecursiveTask<Integer> {  
    //attributes and constructor  
  
    protected Integer compute(){  
        if(hi - low < THRESHOLD) return simpleSum(array, low, hi);  
        Sum left = new Sum(array, low, (hi+low)/2);  
        Sum right= new Sum(array, (hi+low)/2, hi);  
        left.fork();  
        return right.compute() + left.join();  
    }  
}  
  
sum = ForkJoinPool.commonPool().invoke(new Sum(array, 0, array.length));
```

ADTs we've learned

- List
- Stack
- Queue
- Iterator
- Set
- Map
- Priority Queue

What ADT would you use for the diabetes problem?

- A. Stack
- B. Queue
- C. Set
- D. Map
- E. Priority Queue

What ADT would you use for the diabetes problem?

- A. Stack
- B. Queue
- C. Set
- D. Map (food name to blood sugar level)
- E. Priority Queue

What ADT would you use for the Happy Vet problem?

- A. Stack
- B. Queue
- C. Set
- D. Map
- E. Priority Queue

What ADT would you use for the Happy Vet problem?

A. Stack

B. Queue

C. Set

D. Map

E. Priority Queue

What ADT would you use for the rat feeding problem?

- A. Stack
- B. Queue
- C. Set
- D. Map
- E. Priority Queue

What ADT would you use for the rat feeding problem?

- A. Stack
- B. Queue
- C. Set
- D. Map
- E. Priority Queue

What ADT would you use for the mascot selection problem?

- A. Stack
- B. Queue
- C. Set
- D. Map
- E. Priority Queue

What ADT would you use for the mascot selection problem?

- A. Stack
- B. Queue
- C. Set
- D. Map (name to list of arguments)
- E. Priority Queue

What ADT would you use for the sports statistics problem?

- A. Stack
- B. Queue
- C. Set
- D. Map
- E. Priority Queue

What ADT would you use for the sports statistics problem?

- A. Stack
- B. Queue
- C. Set
- D. Map (from name to statistics object)
- E. Priority Queue

What ADT would you use for the character selection problem?

- A. Stack
- B. Queue
- C. Set
- D. Map
- E. Priority Queue

What ADT would you use for the character selection problem?

- A. Stack
- B. Queue (one for subscriber suggestions and one for others)
- C. Set
- D. Map
- E. Priority Queue